

Lab Session 7: The Data Architect (Objects)

Total Estimated Time: 4 - 5 hours **Objective:** Master the creation and manipulation of JavaScript Objects. Learn to write internal methods using `this`, iterate over arrays of objects, and utilize modern ES6 syntax (Destructuring and the Spread operator).

Part 1: Object Foundations & Methods in Node.js (Estimated Time: 1.5 hours)

Let's build core muscle memory for creating objects and methods.

Activity 1: The Book Database

1. Create a file named `library.js`.
2. Create an object called `book` with the following keys: `title` (string), `author` (string), `pages` (number), and `isRead` (boolean).
3. Add a method inside the object called `summarize`.
4. **Task:** The `summarize` method should use the `this` keyword to return a string like: *"The Hobbit by J.R.R. Tolkien is 310 pages long."*
5. `console.log(book.summarize());`
6. Outside the object, change the `isRead` property to its opposite (if true, make it false) using Dot Notation.

Activity 2: The Interactive Bank Account

1. Create `bank.js`.
 2. Create an object called `myAccount`. It needs:
 - A property `owner` (your name).
 - A property `balance` (start at 1000).
 - A method `deposit(amount)`: Adds the amount to the balance and prints *"Deposited \$X. New balance: \$Y"*.
 - A method `withdraw(amount)`: Checks if there is enough money. If yes, subtracts it and prints the new balance. If no, prints *"Insufficient funds!"*.
 3. **Task:**
 - Call `myAccount.deposit(500);`
 - Call `myAccount.withdraw(2000);`
 - Call `myAccount.withdraw(400);`
 - Remember to use `this.balance` inside your methods!
-

Part 3: Modern ES6 Magic in Node.js (Estimated Time: 1 hour)

Time to practice the "clean code" features used heavily in modern frameworks.

Activity 3: Profile Destructuring

1. Create `profile.js`.
2. Paste this complex object:

```
const userProfile = {
  username: "coder99",
  email: "coder99@example.com",
  address: {
    city: "San Francisco",
    state: "CA",
    zip: "94105"
  }
}
```

```
    },  
    hobbies: ["gaming", "hiking", "coding"]  
  };  
};
```

3. Task:

- Without using `userProfile.` anywhere in your `console.log`, use **Object Destructuring** to extract `username`, `email`, and `city`. (Hint for city: you'll have to destructure the `address` object separately, or look up nested destructuring!).
- Print: `"User coder99 from San Francisco can be reached at coder99@example.com."`

Activity 4: The Spread Operator (...) Merging

1. Create `merge.js`.
2. Create an object `playerStats = { health: 100, mana: 50, level: 1 }`.
3. Create an object `playerEquipment = { weapon: "Sword", armor: "Iron" }`.

4. Task:

- Use the spread operator to combine both objects into a brand new object called `fullCharacter`.
- Also, inside that same spread creation, add a brand new property: `class: "Warrior"`.
- `console.log(fullCharacter);`

Part 3: Arrays of Objects & The DOM (Estimated Time: 1.5 hours)

This is how almost all web data is formatted. Let's render a dynamic page!

Activity 5: The Employee Directory

1. Create `directory.html`.
2. Paste this starter code:

```
<!DOCTYPE html>  
<html>  
  <style>  
    body { font-family: sans-serif; }  
    .card { border: 1px solid #ccc; padding: 15px; margin-bottom: 10px; width: 250px;  
background-color: #f9f9f9; }  
    .card h3 { margin: 0 0 5px 0; }  
    .card p { margin: 0; color: #555; }  
    .manager { border-left: 5px solid blue; }  
  </style>  
  <body>  
    <h2>Company Directory</h2>  
    <div id="container"></div>  
  
    <script>  
      // Array of Objects!  
      const employees = [  
        { name: "Sarah Smith", title: "Software Engineer", isManager: false },  
        { name: "John Doe", title: "Project Manager", isManager: true },  
        { name: "Emily Chen", title: "UX Designer", isManager: false }  
      ];  
  
      const container = document.getElementById('container');
```

```

let htmlString = "";

// --- YOUR LOGIC HERE ---
// 1. Loop over the 'employees' array using .forEach()
// 2. For each employee, construct a string of HTML representing a card.
// 3. Add that string to 'htmlString'.
// 4. Challenge: If the employee isManager is true, add the class "manager" to the
card div!

// Example structure for the HTML string:
// <div class="card">
//   <h3>Name Here</h3>
//   <p>Title Here</p>
// </div>

// Finally, inject the HTML to the screen:
// container.innerHTML = htmlString;
</script>
</body>
</html>

```

Activity 6: Destructuring in the DOM

1. In the `directory.html` file from Activity 5, look at the `.forEach` loop you just wrote.
2. Inside the loop, before you write the HTML string, **destructure** the employee object.
 - `const { name, title, isManager } = employee;`
3. Update your HTML template literals to use these clean variables instead of `employee.name` , `employee.title` , etc. Refresh the browser to ensure it still works!

Part 4: The 20-Question Object Master Quiz (Estimated Time: 1 hour)

Section A: Syntax & Properties

1. What symbols are used to create an Object? `[]` , `{}` , or `()` ?
2. The data inside an object is stored as _____ pairs.
3. Given `let car = { color: "red" }` , how do you access the color using Dot Notation?
4. Given `let car = { color: "red" }` , how do you access the color using Bracket Notation?
5. How do you delete the `color` property from the `car` object?

Section B: Methods & this 6. A function that belongs to an object is called a _____. 7. Inside an object's method, what keyword is used to refer to the object itself? 8. True or False: You can store an array inside an object. 9. True or False: You can store an object inside an array. 10. If a method uses `this.price` , but the object does not have a `price` property, what will `this.price` return?

Section C: Modern Syntax (ES6) 11. What is the name of the syntax used to extract properties from an object into distinct variables? (e.g., `const { age } = user;`) 12. What symbol is used for the Spread Operator? 13. If `obj1 = { a: 1 }` and `obj2 = { b: 2 }` , what does `let obj3 = { ...obj1, ...obj2 }` create?

Section D: What's the Output? 14.

```

let dog = { name: "Rex" };
dog.age = 3;

```

```
console.log(dog.age);
```

Output: _____

15.

```
let laptop = { brand: "Dell", isOn: true };  
let key = "brand";  
console.log(laptop.key);
```

Output: _____ (Careful! Dot notation vs variables)

16.

```
let person = {  
  name: "Mia",  
  greet: function() { return "Hi " + this.name; }  
};  
console.log(person.greet());
```

Output: _____

17.

```
let arr = [{id: 1}, {id: 2}, {id: 3}];  
console.log(arr[1].id);
```

Output: _____

18.

```
let original = [1, 2];  
let copy = [...original, 3];  
console.log(copy.length);
```

Output: _____

Section E: Find the Bug 19.

```
let user = {  
  name = "Tom",  
  age = 40  
};
```

Bug: _____

20.

```
let robot = {  
  model: "X-Wing",
```

```
fly: function() {  
  console.log("Model " + model + " is flying!");  
}  
}  
robot.fly();
```

Bug: _____
